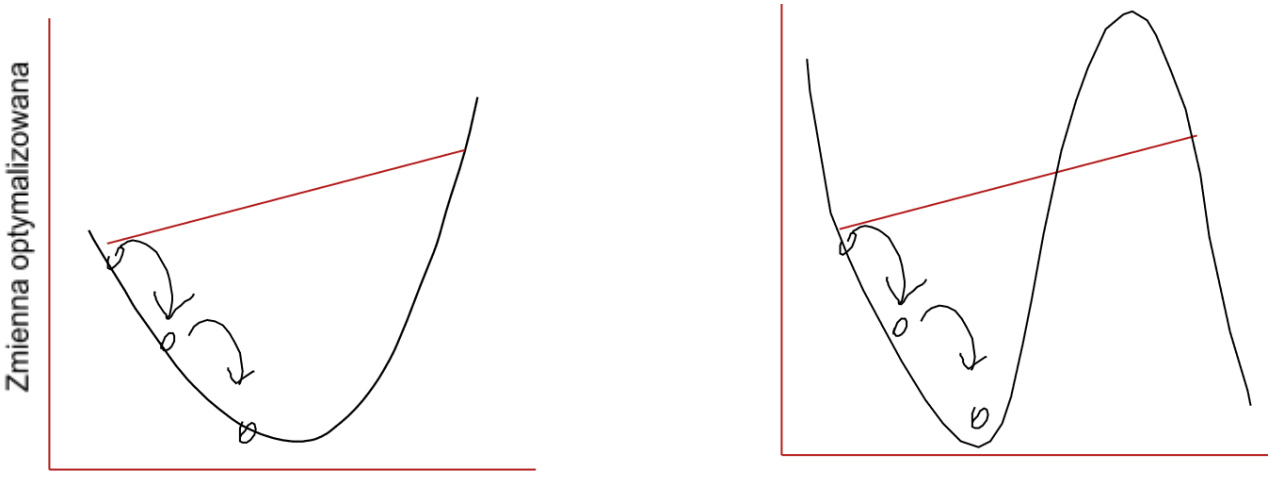
SIECI NEURONOWE – ćwiczenie 2

W drugim ćwiczeniu zajmiemy się uczeniem maszynowym. Wykorzystamy regresję logistyczną – metodę statystyczną która można uznać za formę najprostszej (jednowarstwowej) sieci neuronowej. Model będziemy wykorzystywać do klasyfikacji, czyli chcemy, aby określał prawdopodobieństwo przynależności próbki opisanej wektorem x do właściwej klasy. Do tego celu możemy wykorzystać zbiór przykładów uczących w postaci par wejście-klasa.

Klasyfikator liniowy interesuje nas, ponieważ trenowanie go jest przykładem optymalizacji wypukłej. Zarówno funkcja klasyfikatora, jak i funkcja kosztu są wypukłe, co daje nam kształt w przestrzeni optymalizacji, po którym optymalizator oparty o gradient może zawsze znaleźć drogę do globalnego minimum. Poniższy rysunek poglądowy obrazuje sytuację:



Po lewej, na funkcji wypukłej, optymalizator gradientowy zawsze znajdzie drogę do globalnego minimum. Po prawej, na funkcji niewypukłej, nie ma takiej gwarancji.

Sieci neuronowe są problemem niewypukłym. Uczenie oparte o gradient pozostaje jednak efektywną metodą uczenia (lub bardziej realistycznie, *jedynym* wyborem, bo w tak gigantycznej przestrzeni parametrów potrzebujemy *jakiegoś* kierunku optymalizacji). Ale żeby zrozumieć problemy, które pojawią się w optymalizacji niewypukłej, najpierw zobaczymy jak przebiega uczenie (względnie) bez problemów.

Możemy zdefiniować wyjście jako:

$$p(x)=\sigma(Wx+b)$$

Gdzie x to wektor wejściowy, W i b to parametry modelu natomiast funkcja sigma to

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Jest to wygodna funkcja dająca wyniki przedziale [0,1] dla każdego rzeczywistego n,

przy okazji rosnąca i różniczkowalna.

Aby wyuczyć model w jakikolwiek sposób, potrzebujemy zdefiniowanego zadania optymalizacji, a do tego potrzebujemy funkcji kosztu którą będziemy minimalizować. Korzystamy z entropii krzyżowej, dla przykładu (x,y) w klasyfikacji binarnej:

$$L=-y \ln p(x)-(1-y)\ln(1-p(x))$$

(y będzie zawsze zerem lub jedynką, czyli tylko jeden ze składników będzie niezerowy. Wartość sumujemy po wszystkich przykładach w zbiorze danych, suma będzie pominięta w wzorach.)

Zaletą takiego kosztu jest fakt, że jej pochodna po wagach modelu jest bardzo prosta:

$$\frac{\partial L}{\partial w_i} = -\ (y\ -\ p(x))x_i$$

Optymalizacja modelu będzie polegała na wykonywaniu niewielkich kroków w kierunku wyznaczonym przez gradient – pochodne cząstkowe po wszystkich wagach – w pętli uczenia, aż model osiągnie zbieżność. Innymi słowy, aktualizujemy wagi według:

$$w_i \Leftarrow w_i - \alpha \frac{\partial L}{\partial w_i}$$

Gdzie alfa to pewien współczynnik uczenia, hiperparametr który musimy ustawić z góry.

Zadaniem na zajęcia 2 jest implementacja działającego na zbiorze *heart disease* modelu klasyfikacji, przy czym, elementy które zależy nam żeby przebadać to mierzalna jakość klasyfikacji (metryki F-score, precision, recall) oraz szybkość zbieżności. Zależy nam na określeniu wpływu na zbieżność

- współczynnika uczenia
- rozkładu inicjalizacji parametrów
- liczby epok/warunku stopu

Przy czym dla tego ostatniego, warto skorzystać z tzw. krzywej uczenia: wykresu funkcji kosztu i metryk w zależności od iteracji uczenia.

A obserwacje których chcemy dokonać mają nam głównie pokazać na pytanie: czy klasyfikator liniowy zawsze zbiega do pożądanego przez nas globalnego minimum? Czy mogą jednak pojawić się problemy związane ze złą parametryzacją i czy te problemy kompletnie uniemożliwiają zbieżność/dobrą jakość wyniku, czy tylko przedłużają uczenie?